

Administración de dispositivos de entrada – salida

Administración de E/S

El sistema de E/S es el método implementado para controlar diversos dispositivos y tiene por objetivo proporcionar la interfaz más sencilla al resto del sistema.

Los sistemas operativos buscan dos objetivos a lograr en la administración de los elementos de E/S que son:

Eficiencia

Consiste en distraer lo menos posible al procesador con tareas de E/S.

Generalidad

Debido a que existe una gama de elementos de E/S de características totalmente diferentes se debe tratar de gestionar a todos los dispositivos de E/S de manera uniforme, debido a la forma en que el sistema operativo gestiona a los dispositivos, y a como los procesos ven a los dispositivos. Para lograr esta generalización de los dispositivos de E/S se puede emplear un enfoque jerárquico y modular para el diseño de las funciones de E/S.

Varios dispositivos de E/S utilizan un mismo conjunto de cables denominado bus, sobre el cual hay diferentes puertos. Los buses se manejan con protocolos predeterminados y cada controlador envía su pulso eléctrico que es diferente al de otro controlador.

- Se envía una señal por cable o aire.
- Se comunica a través de un punto de conexión denominado puerto.

Drivers (software)

Consiste en un software que realiza la interfaz entre los dispositivos de E/S y las aplicaciones.

Características de los drivers

La interfaz tiene por objeto que el sistema operativo trate a los dispositivos de E/S en forma uniforme, sea cual sea el dispositivo.

La estandarización la logra el driver, ocultando al sistema operativo la diferencia que existe entre cada controlador.

Clasificación de dispositivos de E/S

Los diferentes dispositivos de E/S se pueden agrupar en las siguientes clases:

- Generales: discos, unidad de cinta, sensores, controladores, etc.
- De transmisión: permiten la comunicación con dispositivos lejanos: adaptadores de líneas digitales, módem, placas de red, etc.
- De enlace con el usuario: terminales de video, teclado, pantalla, mouse, impresora, etc.

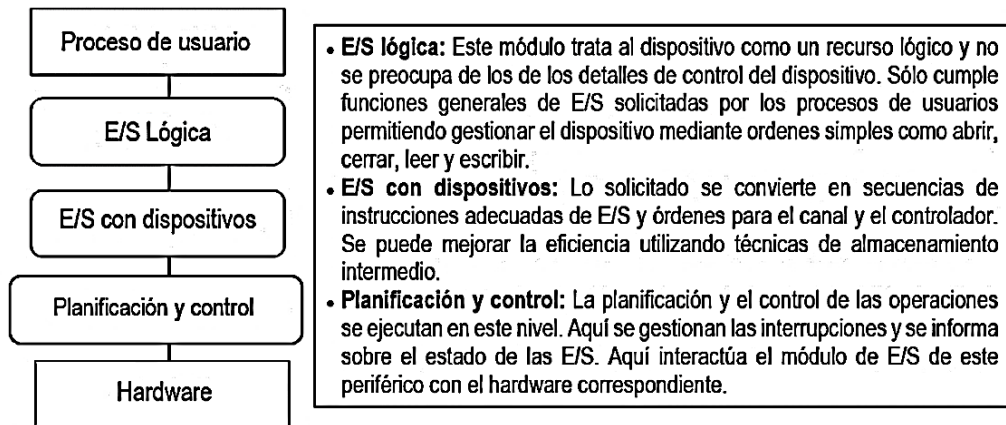
Existen muchas diferencias entre las distintas clases, como por ejemplo la velocidad de transmisión de los datos, la utilidad que se le da a cada uno de los dispositivos, la complejidad del control, la unidad de transferencia, la representación de los datos, las condiciones de error, etc.

Una de las partes más complicadas del sistema operativo es aquella que se encarga del manejo de la E/S. La E/S es un elemento clave en el rendimiento del sistema operativo. Las velocidades del procesador son muy dispares con respecto a las de E/S e incluso a las de memoria principal, aunque estas están mejorando sensiblemente usando uno, dos o más niveles de caché interna.

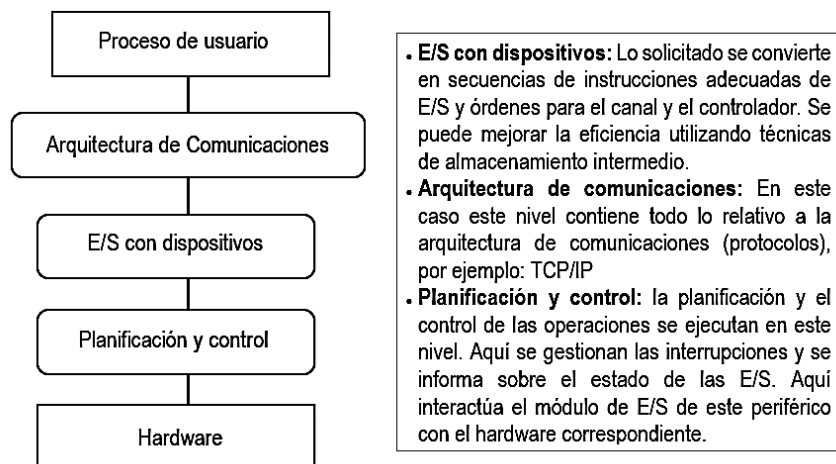
Estructura lógica de las funciones de E/S

Una de las formas más eficientes de estructurar los sistemas operativos con tareas complejas es la estructura jerárquica donde las partes de menor nivel son las que mantienen contacto con el hardware y manejan tiempos de intercambio muy cortos. Por otra parte, estas capas de menor nivel ofrecen servicios a las capas de nivel inmediatamente superior. Esta filosofía es muy útil en el manejo de los sistemas de E/S y la aplicación de esta filosofía conduce a una organización jerárquica como las que se muestran en los siguientes ejemplos:

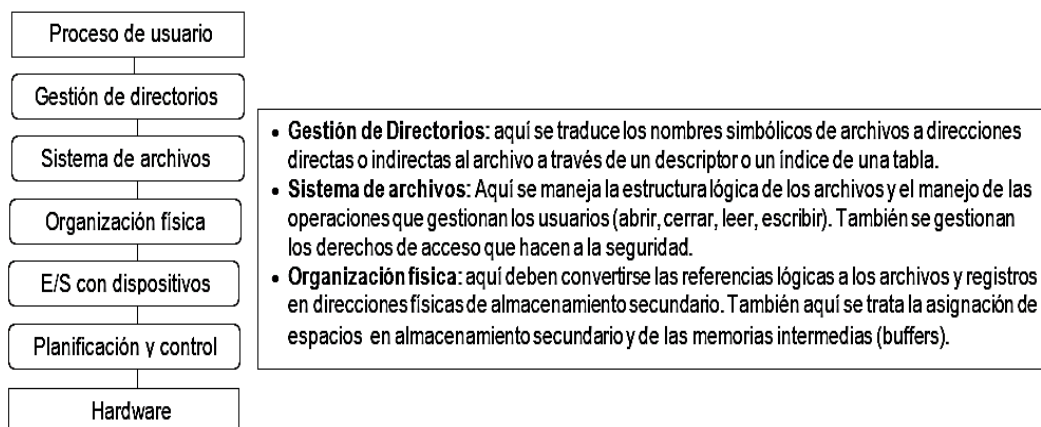
Caso de una E/S sobre un dispositivo de periférico local



Caso de una E/S a través de un puerto de comunicaciones



Caso de una E/S a través de un sistema de archivos



Opciones para realizar E/S

1. Procesador controla directamente los dispositivos periféricos.
2. El procesador utiliza E/S programada sin interrupciones usando un controlador o módulo de E/S. En este esquema el procesador ejecuta una orden de E/S pedida por un proceso y luego espera a que termine la operación antes de seguir.
3. Igual al punto 2, pero el procesador continúa la ejecución de las instrucciones siguientes (ya sean del mismo proceso en el caso que no sea necesario esperar la terminación de la E/S o de otro proceso) a la espera de la interrupción por parte del módulo de E/S cuando completa su trabajo.
4. El módulo de E/S recibe el control de la memoria a través del DMA, el cual transfiere un bloque de datos sin intervención del procesador, excepto al comienzo y el final de la transferencia.

Acceso directo a memoria (DMA)

El módulo de DMA es un procesador especializado capaz de imitar al procesador central para realizar el control de las operaciones de E/S. Tiene como objetivo no sobrecargar al procesador con esas tareas y de esta manera evitar que la CPU deba vigilar los bits de estado del dispositivo y alimentar un registro del controlador byte a byte. Luego, la unidad de DMA será capaz de tomar el control de todo el sistema de E/S para transferir datos desde y hacia la memoria principal a través del bus del sistema.

Para poder realizar su tarea el módulo de DMA está conformado, entre otros, por los siguientes componentes:

- Registro de datos.
- Registro de estado o registro de control.
- Registro de direcciones.
- Controlador del dispositivo.
- Unidad de control.

La técnica del DMA básicamente trabaja de la siguiente manera:

- 1) Cuando el procesador, bajo el control del sistema operativo, desea leer o grabar un bloque de datos en un dispositivo externo, emite una orden indicándole al driver del dispositivo que debe transferir C datos desde/hacia el disco a/desde la

dirección X de memoria. Esta orden contendrá los siguientes datos que se le enviarán al DMA:

- Dirección del dispositivo involucrado.
- La posición inicial de memoria a partir de donde se lee o se escribe.
- Cantidad de palabras a leer o escribir.
- Tipo de operación (lectura o escritura).

Es trabajo de la CPU hacerle llegar al DMA estos datos que forman el bloque de control de DMA para que pueda realizar las transferencias deseadas, delegándole así el control de la operación de E/S al módulo de DMA, que se encargará de ella. Una vez que el procesador pasó todos estos datos continúa con otro trabajo.

- 2) Una vez recibido el bloque de control, el módulo de DMA solicita al procesador el acceso a la memoria principal, mediante la activación de la correspondiente señal de control.
- 3) La CPU responde, mediante una señal de control, concediendo la solicitud.
- 4) El módulo de DMA realiza los accesos a memoria principal, activando las señales de control y generando las direcciones correspondientes.
- 5) Cuando el módulo de DMA ha terminado devuelve a la CPU el control de los buses y de las señales de control, desactivando la señal de petición del bus.
- 6) Cuando el DMA completó su tarea, es decir terminó con la operación de E/S, le envía una señal de interrupción al procesador y éste recupera el uso de los buses. De esta manera se observa que el procesador sólo está involucrado en la tarea de E/S al principio y al final de la transferencia.

El DMA se puede configurar de tres maneras diferentes las cuales son:

DMA independiente

El DMA actuando como un procesador suplente y realiza todas las tareas de E/S a través del BUS del sistema. Esta es una configuración barata pero ineficiente porque la transferencia de cada palabra consume ciclos del BUS del sistema.

Si se combinan las funciones del módulo de E/S con las del DMA se acorta sustancialmente la cantidad de ciclos de BUS del sistema requeridos. Esto se puede lograr de dos maneras diferentes:

DMA integrado

Tener más de un DMA conectado al bus del sistema y donde cada DMA agrupa una serie de elementos de E/S de características equivalentes.

DMA con bus de E/S

Tener un DMA conectado al bus del sistema y los elementos de E/S se conectan al DMA a través de un BUS de E/S.

Contenido de los puertos de E/S del controlador

- Si la orden ya se ejecutó, el puerto queda en estado libre.
- Si en el registro de entrada hay datos para leer, el puerto queda en estado de ocupado.
- Si hubo un error de dispositivo se enciende el bit de estado correspondiente (ERROR).
- Se encenderá el bit de lectura o escritura, según la llamada al sistema que se haya realizado.
- Cuando el controlador está atendiendo a otro proceso el puerto está ocupado (no listo) y cuando no está atendiendo a nadie está libre (listo).
- Los registros de salida y entrada son la antesala de la entrada o salida de datos.

Métodos de interacción entre el dispositivo y el controlador

Son los siguientes:

Escrutinio (POLLING)

Consiste en leer el registro de estado tantas veces como sea necesario, hasta que lo encuentre libre. Es una operación eficiente, pero se torna ineficiente cuando tiene que chequear mucho tiempo ocupado.

Interrupciones

La CPU tiene líneas de control llamadas líneas de solicitud de interrupción (IRQ), por cada dispositivo. La CPU después de ejecutar cada instrucción verifica las IRQ y detecta si hubo alguna interrupción.

Se pueden diferenciar los siguientes tipos de interrupciones:

- Enmascarables: la CPU las desactiva antes de iniciar un proceso crítico y luego la vuelve a activar.
- No enmascarables: están reservadas a sucesos de alta prioridad, por ejemplo: error de memoria.

Todas ellas presentan las características siguientes:

- Capacidad de postergar las interrupciones durante procesos más críticos.
- Disponer de IRQ de múltiples niveles, para que el sistema operativo distinga entre interrupciones de alta y baja prioridad.
- Cada IRQ se Complementa con una rutina.

En los sistemas actuales existe el vector de interrupciones que almacena la dirección de la rutina que corresponde a cada IRQ.

Almacenamiento intermedio (búfer) de las E/S

Para evitar la espera del movimiento de datos es conveniente realizar el movimiento antes de que se hagan las peticiones correspondientes, almacenando los datos en una memoria de almacenamiento más rápido llamada búfer. Luego, es muy común que, para aumentar la eficiencia de los procesos de E/S, se utilice un espacio de memoria intermedia. La técnica consiste en que mientras un bloque se sacó y se entregó al procesador para que realice su tarea de actualización, el dispositivo de E/S almacena otro bloque en un espacio de memoria intermedia destinado a tal efecto.

Existen diferentes métodos para el uso de memorias intermedias, a saber:

Almacenamiento intermedio sencillo o único (búfer único)

En este esquema, toda vez que un proceso realice un pedido para ejecutar una operación de E/S, el sistema operativo le asignará un búfer, dentro de su área de memoria real, para la realización de la operación.

Almacenamiento intermedio doble (búfer doble)

Esta técnica se basa en la idea de asignarle a un proceso no un único búfer para utilizar durante una operación de E/S sino dos. De esta forma, mientras un proceso transfiere datos hacia o desde un búfer, el sistema operativo está vaciando o llenando el otro.

Almacenamiento intermedio circular (búfer circular)

Ante la necesidad de un proceso de realizar ráfagas de E/S rápidas, las dos técnicas anteriores se muestran deficientes. Por este motivo, frecuentemente se utilizan más de dos búferes para solucionar el problema, llamando al conjunto de búferes empleado: búfer circular.

Con el almacenamiento intermedio se logra solucionar problemas de picos de demandas de E/S, aunque no de forma permanente porque eventualmente todos los búferes se llenarán y entonces el proceso deberá esperar a terminar de procesar cada grupo de datos. Pero, en un entorno de multiprogramación es una técnica que puede aumentar la eficiencia del sistema operativo como así también el rendimiento individual de cada proceso.

Interfaz de los dispositivos de E/S

Es posible clasificar las interfaces de los dispositivos de E/S en dos clases: interfaz por bloques e interfaz de flujo de caracteres.

Interfaz por bloques

Su uso es común en dispositivos orientados a bloques, como los discos, cintas u otros dispositivos con datos agrupados de tamaño fijo que son transferidos bloque a bloque.

Interfaz de flujo de caracteres

Su aplicación es común en dispositivos orientados al flujo de caracteres como las impresoras, el teclado, el mouse, el modem, etc., donde los datos que se reciben o se emiten son transferidos como un flujo de bytes, sin presentar una estructura de bloques.

Servicios del núcleo del sistema de E/S

- Planificación de E/S: trata de planificar los accesos para lograr el menor tiempo de respuesta posible.
- Utilizar espacio en memoria principal o disco, usando técnicas de búferes, caché y SPOOL.

- **Búfer:** área de memoria donde se almacena información mientras se transfiere entre dispositivos, o entre dispositivo y aplicación. Su uso se debe a tres razones:
 - Diferencia de velocidades entre productor y consumidor.
 - Adaptador de dispositivos que tienen tamaños de almacenamiento.
 - Garantizar que la versión de datos que se toma es la que existía en el momento en que la aplicación hizo la llamada al sistema.
- **Caché:** es una región de memoria rápida que contiene copia de datos, con el objeto de agilizar la gestión
- **SPOOL (Simultaneous Peripheral Operation on Line):** es un búfer que contiene salidas para un dispositivo determinado, por ejemplo: impresoras. El SPOOL es manejado por un proceso llamado Demonio y permite que el administrador visualice la cola de impresión y eventualmente pueda realizar operaciones sobre ella.

Discos

Parámetros de rendimiento

El crecimiento de las velocidades de procesamiento y de acceso a memoria principal ha dejado muy atrás los tiempos de acceso a disco.

Los parámetros de rendimiento de disco son:

Tiempo de búsqueda

Es el tiempo que se demora en posicionar la cabeza lectora/grabadora sobre la pista deseada. A su vez este tiempo se compone de dos tiempos diferentes:

- Tiempo de arranque inicial.
- Tiempo que tarda en recorrer las pistas.

Retardo o latencia de giro

Es el tiempo de giro del disco hasta que la cabeza lectora/grabadora se posiciona al inicio del sector o pista correspondiente.

Tiempo de acceso

Es la suma de los dos tiempos anteriores.

Tiempo de transferencia

Una vez posicionado en la pista y al inicio del sector o pista se procede a realizar la transferencia real de los datos y el tiempo que esto demanda se denomina tiempo de transferencia.

Tiempo de espera

Además de estos tiempos se puede adicionar un tiempo más que es el que el proceso está en cola hasta que se le asigne el acceso al disco porque hay otro proceso en ejecución sobre un único canal de acceso a disco.

Valores de referencia de tiempos en discos

- El tiempo de búsqueda en disco actual es de 5 a 10 mseg.
- El retardo de giro es de 5.400 a 10.000 rpm, la última equivale a una vuelta cada 6 mseg. Por lo tanto, el retardo medio del giro es de 3 mseg.
- Cálculo del tiempo de transferencia:

$$T = \frac{b}{r \times N}$$

- Donde:
 - $T \rightarrow$ tiempo de transferencia.
 - $b \rightarrow$ número de bytes a transferir.
 - $N \rightarrow$ número de bytes por pista.
 - $r \rightarrow$ velocidad de rotación en revoluciones por segundo.
- Luego:

$$T_a = T_s + \frac{1}{2 \times r} + \frac{b}{r \times N}$$

- Dónde:
 - T_s es el tiempo medio de búsqueda.
 - El segundo término es el tiempo medio de latencia de giro.
 - Y el tercer término es el tiempo de búsqueda.

Ejemplo de comparación de tiempos

Existen dos operaciones de E/S que confían en los siguientes valores medios:

- Tiempo medio de búsqueda conocido 10 mseg.
- Velocidad de rotación 10.000 rpm.
- Sectores de 512 bytes.
- 320 sectores por pista.

Se desea leer un archivo de 2560 sectores:

Tamaño del archivo = $2560 \times 512 = 1.310.720$ bytes = 1280 KB = 1,25 MB

Estimar el tiempo total de transferencia, suponiendo que el archivo está almacenado en forma contigua.

Si está almacenado en forma contigua utilizará:

$$\text{Cantidad de pistas} = \frac{2560 \text{ sectores}}{320 \text{ sectores}} = 8 \text{ pistas}$$

Al estar almacenado en forma contigua se trata de una organización secuencial simple y el tiempo de lectura de la primera pista será de:

Búsqueda media	10 mseg.	
Retardo de giro	3 mseg.	Por la velocidad ya definidas
Lectura de 320 sectores	<u>6 mseg.</u>	
Total	19 mseg.	

Suponer que las pistas restantes se pueden leer sin tiempo de búsqueda y sólo se considera el retardo de giro de las pistas sucesivas, por lo tanto, cada pista sucesiva se lee en 3 mseg. + 6 mseg. = 9 mseg.

Para leer el archivo completo se tardará...

$$\text{Tiempo Total} = 19 + 7 \times 9 = 82 \text{ mseg} = 0,082 \text{ seg}$$

En cambio, si el acceso al archivo ahora es aleatorio a cada pista el tiempo será:

Búsqueda media	10 mseg.	
Retardo de giro	3 mseg.	Por la velocidad ya definidas
Lectura de 1 sector	<u>0,01875 mseg.</u>	
Total	13,01875 mseg.	

Lo cual muestra que lo que más incide en el tiempo total es el tiempo de búsqueda, es por ello por lo que el sistema operativo debe actuar sobre estos tiempos definiendo diferentes algoritmos de planificación de búsqueda en discos.

Planificadores de disco

Un planificador de disco examina las direcciones de los sectores que tiene cada petición pendiente en la cola del disco, y las reordena de tal manera que éstas puedan ser servidas con un mínimo de movimientos mecánicos. La planificación de disco debe intentar maximizar el número de peticiones servidas por unidad de tiempo, y minimizar el tiempo de respuesta promedio, eliminando desperdicios de tiempo al efectuar largas búsquedas.

Las dos clases más comunes de planificadores de disco son la optimización de búsqueda y la optimización rotacional. Debido a que los tiempos de búsqueda son mayores que los tiempos de latencia, la mayoría de los algoritmos de planificación se centran en optimizar el tiempo de búsqueda. La optimización rotacional solo se utiliza bajo condiciones de carga muy pesada, y puede ser aplicada tanto en discos de cabezas móviles como de cabezas fijas. La mayoría de los controladores de disco mantienen una tabla o cola de peticiones, indexada por número de cilindro, con todas las peticiones pendientes para cada cilindro. A continuación, se describen los principales algoritmos utilizados en la optimización de búsqueda.

FCFS (First Come, First Served)

Es la forma más sencilla de planificación, se brinda el servicio por orden de llegada. Suponer que existe una serie de requerimientos de pistas como la siguiente y que se comienza en la pista 100: 55 – 58 – 39 – 18 – 90 – 160 – 150 – 38 – 184. La primera petición que llega a la cola de peticiones es la primera en ser servida, y así sucesivamente, de forma que ninguna petición será desplazada, o postergada, por la llegada de otra petición con una prioridad mayor.

Este algoritmo da un patrón de búsqueda al azar si las peticiones se encuentran uniformemente distribuidas en el disco. El algoritmo genera tiempos de respuesta aceptables cuando se tiene una cantidad de peticiones pequeña, pero, cuando la cantidad de peticiones de la cola aumenta considerablemente, el rendimiento del disco decrecerá porque el brazo del disco tenderá a viajar de un extremo al otro del disco, dando como resultado tiempos de respuesta largos.

SSTF (Shortest Seek Time First)

Este algoritmo sirve la petición que produzca el menor tiempo de búsqueda; es decir, aquella petición que da por resultado el movimiento del brazo más corto con respecto a la posición actual del mismo. Por lo tanto, el orden en que llegan las peticiones no se mantiene. Suponer entonces que se tiene la misma serie de peticiones de pistas que en el caso anterior y que se comienza en la pista 100. Luego la serie sería atendida en el siguiente orden: 90 – 55 – 58 – 39 – 38 – 18 – 160 – 150 – 184.

Este tipo de algoritmo presenta un patrón de búsqueda muy concentrado en una sola región, principalmente las pistas intermedias del disco, por lo que las pistas de los extremos interno y externo reciben una menor atención. Tiene la ventaja de que da un buen número de peticiones servidas por unidad de tiempo, y una media de tiempo de respuesta baja para cargas moderadas, por lo que es muy recomendable para procesamiento por lotes. Sin embargo, para sistemas de tiempo compartido no es recomendable debido a que causa tiempo de respuestas largos para las peticiones que están en las pistas de los extremos. En resumen:

- Atiende las solicitudes cuyo tiempo de búsqueda es más corto.
- A pesar de tener una mejora sustancial con respecto a FCFS, no es óptimo.
- Este algoritmo puede producir *starvation* o inanición.

SCAN

En este algoritmo, la cabeza empieza en uno de los extremos del disco (por ejemplo, el exterior), y empieza a moverse hacia el extremo opuesto (el interior), dando cumplimiento a las peticiones que se refieren a cada pista que va encontrando en su camino. Al llegar el extremo opuesto, la cabeza invierte el sentido de dirección y emprende el viaje de vuelta, atendiendo ahora a todas las peticiones que se deben realizar en las pistas señaladas en esa trayectoria. Así, por ejemplo, si una petición llega a la cola del disco en el instante en que la cabeza pasa delante de la pista requerida por esa

petición, será servida casi inmediatamente; mientras que, si la petición llega justo después que pase la cabeza por la pista, entonces tendrá que esperar hasta que la cabeza emprenda el viaje de vuelta y pase por la pista solicitada. El viaje de vuelta puede ser iniciado no solamente cuando se alcanza el extremo del disco, sino también cuando ya no haya más peticiones en esa dirección en cuyo caso adquiere el nombre de planificación LOOK.

Este algoritmo produce un buen número de peticiones servidas por unidad de tiempo, así como una buena medida de tiempos de respuesta. Su característica principal es que evita las discriminaciones que se tienen en el algoritmo SSTF, por lo que se lo recomienda para los sistemas de tiempo compartido. Tiene la desventaja de que los cilindros de los extremos son visitados con menos frecuencia, pero esto no es tan marcado como en el algoritmo SSTF.

En resumen, el algoritmo SCAN presenta las características siguientes:

- El brazo del disco se mueve desde un extremo al otro del disco y luego hace el recorrido inverso, en su recorrido va atendiendo los requerimientos que encuentra inmediatamente adelante.
- Si se supone una distribución de requerimientos constante, cuando regresa lo hace a un tiempo muy breve donde podría no encontrar requerimientos.
- Suponiendo que se tiene la misma serie de requerimientos de pistas que antes y que se comienza en la pista 100, la serie sería atendida en el siguiente orden: 150 – 160 – 184 – 199 – 90 – 58 – 55 – 39 – 38 – 18.

SCAN de N pasos

Una variante del algoritmo SCAN es el algoritmo SCAN de N pasos. La diferencia estriba en que para este algoritmo la cola de peticiones es dividida en varias colas todas con la misma capacidad N. Las peticiones de cada una de estas colas serán atendidas según el algoritmo SCAN y sólo se atenderá una cola por vez. Esto significa que mientras se están atendiendo las peticiones de una cola, todas las peticiones nuevas se agregan a otra de las colas y serán atendidas posteriormente.

C-SCAN

Este algoritmo es otra variante más del algoritmo SCAN. C-SCAN mueve el brazo de un extremo a otro igual que SCAN, atendiendo las peticiones que encuentra en su trayectoria. Al llegar al extremo opuesto, el brazo regresa rápidamente al extremo del que

partió, sin servir ninguna petición en su regreso. Por lo tanto, C-SCAN trata el disco como si éste fuera circular; es decir, con la última pista adyacente a la primera. C-SCAN elimina completamente la distinción entre cilindros intermedios y cilindros externos o internos y proporciona un tiempo de respuesta más parejo para todas las peticiones.

Resumiendo:

- Al igual que el SCAN mueve la cabeza de un extremo al otro, pero a la vuelta regresa sin leer.
- Si se tiene la misma serie de requerimientos de pistas que antes, y se supone que se comienza en la pista 100, entonces, la serie sería atendida en el siguiente orden: 150 – 160 – 184 – 199 – 0 – 18 – 38 – 39 – 55 – 58 – 90.

FSCAN

Otra variante del algoritmo SCAN en el cual se utilizan 2 colas de peticiones. El algoritmo comienza a atender todas las peticiones de una de las colas mientras la otra, que inicialmente estaba vacía se va llenando con nuevas peticiones, que recién serán procesadas cuando se hayan atendido todas las peticiones de la primera cola.

LOOK

Similar al SCAN, pero llega al extremo del último requerimiento e invierte el movimiento. Suponer que se tiene la serie de requerimientos de pistas mencionada anteriormente y que se comienza en la pista 100, luego, la serie sería atendida en la siguiente secuencia: 150 – 160 – 184 – 90 – 58 – 55 – 39 – 38 – 18.

C-LOOK

Una variante del algoritmo C-SCAN es el C-LOOK, el cual se comporta igual que el C-SCAN excepto que, al irse moviendo hacia el extremo opuesto, si ya no hay más peticiones que servir en el resto de las pistas, entonces emprende el regreso. Suponiendo que se tenga la serie de requerimientos de pistas ya utilizada en los casos anteriores, y que se comienza en la pista 100, entonces, la serie de requerimientos sería atendida en el siguiente orden: 150 – 160 – 184 – 18 – 38 – 39 – 55 – 58 – 90.

SLTF (Shortest Latency Time First)

En este algoritmo cuando el brazo llega a un cilindro en particular (usando cualquier algoritmo de optimización de búsqueda), se examinan todas las peticiones pendientes para ese cilindro y se sirve primero aquella que tiene el tiempo de latencia más pequeño. Esta estrategia se ha usado por muchos años en discos de cabezas fijas ya que estos tienen un tiempo de búsqueda despreciable. Además, es fácil de implementar.

En general, la planificación del disco mejora mucho la eficiencia del sistema; sin embargo, existen ciertos casos particulares en los que es más adecuado aplicar simplemente un algoritmo FCFS. Por ejemplo, cuando no se tienen distribuciones uniformes como en los casos de archivos secuenciales muy largos, ya que los registros están todos contiguos y agrupados en una cierta zona del disco. Esto hace innecesario el algoritmo, ya que sólo existe una sola petición a través de un gran número de cilindros contiguos.

Otro caso problemático para aplicar la planificación de disco son los archivos secuenciales indexados tales como ISAM, ya que el archivo de datos puede estar almacenado en cilindros diferentes a los de los índices. Esto hace que se tengan que realizar búsquedas de un extremo a otro, rompiendo con el algoritmo de planificación. Una solución para evitar romper con la planificación de disco es diferir el acceso al archivo de datos hasta que se pase por los cilindros correspondientes.

Cuando se tiene una cola de peticiones de servicio muy larga es necesario considerar la optimización rotacional para que junto con la optimización de búsqueda puedan manejar adecuadamente las peticiones. La optimización rotacional se basa en que, al tener una cola de peticiones de servicio muy larga, la probabilidad de que ocurran referencias al mismo cilindro aumenta, por lo que se puede utilizar un algoritmo que optimice esas peticiones. Dada la complejidad de todos estos factores, resulta muy conveniente que el módulo de planificación sea independiente del sistema operativo a fin de poder sustituirlo en cualquier momento, por este motivo los fabricantes de discos suelen incluir los algoritmos de planificación en su hardware a través de la controladora de disco.

En conclusión, en base a experiencias obtenidas, se puede decir que la mejor política de planificación de discos se puede manejar en tres etapas: para una cantidad de peticiones pequeña, la planificación SCAN es la mejor; mientras que para cantidades medias y grandes la planificación C-SCAN genera los mejores resultados. Para el caso de cantidades extremadamente grandes, el algoritmo C-SCAN con optimización rotacional produce resultados efectivos.

En definitiva:

- La elección del algoritmo depende de la cantidad y tipo de solicitudes:
 - SSTF tiene un atractivo natural, pero no funciona cuando la demanda es intensa, pues puede caer en un estado de inanición.
 - SCAN y C-SCAN funcionan mejor en demandas intensas.
- Dependerá también del método de asignación de archivos.
 - Asignación contigua genera solicitudes cercanas entre sí en el disco.
 - Asignación enlazada puede incluir bloques muy dispersos.
- Y también depende de la ubicación de directorios y bloques de índices.

Caché de disco

El término de memoria caché se utiliza inicialmente para representar una memoria pequeña de muy rápido acceso y que se sitúa entre el procesador y la memoria principal. El mismo concepto se lo puede aplicar al caché de disco, como la memoria intermedia entre la memoria principal y ciertos sectores de disco.

Se debe tener en cuenta que cuando se mueve información del disco, se mueven varios bloques que se almacenan en el caché, pero luego se mueve sólo uno de ellos y el resto se almacenan en el caché y al solicitarse otros bloques que por cercanías ya están en cache y no se hace necesario leerlos nuevamente desde el disco.

La estrategia de reemplazo del bloque en el cache puede ser:

- LRU (*Least Recently Used*: menos recientemente utilizado): seleccionará para su reemplazo al bloque de caché que menos se haya utilizado.
- LFU (*Least Frequently Used*: menos frecuentemente utilizado): este utiliza un contador de uso para detectar el bloque menos frecuentemente utilizado. El problema que tiene es que si se accede a pequeñas partes aisladas dentro de un mismo bloque dará frecuentes usos y en realidad no es tan así.

Administración de espacio libre en disco

Son utilizados los siguientes métodos:

Vector de bits

- Cada bloque de disco se representa con un bit, el cual tendrá el valor 0 si dicho bloque está libre y el opuesto si está ocupado.
- Se necesitan tantos bits como bloques tenga el disco.
- Es sencillo y eficiente para encontrar n bloques libres contiguos en el disco.

Lista enlazada

- Se enlazan en una lista todos los bloques libres manteniendo un apuntador al primer bloque libre.
- Cada bloque libre apunta al siguiente bloque libre.

Agrupamiento

- Se almacena en el primer bloque las direcciones de n bloques libres.
- Los primeros $n-1$ bloques están libres y el último contiene la dirección de otro bloque que contiene las direcciones de los siguientes n bloques libres.

Recuento

- Se mantiene la dirección del primer bloque libre y un recuento que es la cantidad de bloques contiguos a ese que están libres.
- Se utiliza cuando la asignación es contigua.

RAID (Redundant Array of Independent Disk)

RAID tiene dos objetivos diferentes que son:

- Reemplazar unidades de disco de gran capacidad por unidades múltiples de menor capacidad y de esta manera distribuir los datos en diferentes discos y por ende se poder acceder simultáneamente a diferentes datos.
- Utilizar información de paridad almacenada para recuperar datos perdidos en caso de falla de un disco.

Hay 7 niveles de RAID que van de RAID 0 a RAID 6, que no implican relación jerárquica pero sí tienen tres características comunes:

- 1) Es un conjunto de unidades físicas de disco vistas por el sistema operativo como una sola unidad lógica.
- 2) Los datos están distribuidos a través de las unidades físicas.
- 3) La capacidad del disco redundante se utiliza para guardar información de paridad que garantiza la recuperabilidad de datos en caso de fallo de un disco físico.

Los niveles de RAID 2 y 4 no están disponibles para su comercialización.

RAID 0

Cumple sólo con el primer objetivo de RAID; es decir, reemplazar unidades de disco de gran capacidad por unidades múltiples de menor capacidad con el objetivo de distribuir los datos en diferentes discos y por ende poder acceder simultáneamente a diferentes datos y aumentar así la velocidad de acceso. Por el contrario, no cumple con la redundancia para información de paridad.

RAID 1

Difiere de los niveles RAID 2 a 6, en este caso la redundancia se consigue sólo por duplicar todos los datos, sin utilizar ningún cálculo de paridad de paridad. Tiene varios aspectos positivos:

- Una solicitud de lectura la puede atender cualquiera de los dos discos.
- Si bien la solicitud de escritura se debe satisfacer en ambos discos esta tarea se puede realizar en paralelo.
- La recuperación del fallo es sencilla, pues cuando una unidad falla, los datos están accesibles en forma inmediata desde la otra unidad.

Sus aspectos negativos son:

- Su costo, pues requiere duplicar la capacidad de almacenamiento respecto de la que realmente esta guardada.

RAID 2

Presenta las siguientes características:

- Utiliza el espacio redundante para realizar para realizar cálculos de paridad.
- Trabaja con el código de *hamming*, que es capaz de corregir errores de un solo bit y detectar errores de dos bits.

- Trabaja con particiones muy pequeñas.
- Su costo es muy elevado, más que el del RAID 1.
- El número de discos redundantes es proporcional al logaritmo del número de discos de datos que posee.

RAID 3

Se organiza en forma similar al RAID 2 pero sólo requiere un disco redundante. Además, emplea acceso paralelo con datos distribuidos en pequeñas bandas.

RAID 4

Del RAID 4 al 6 usan una técnica de acceso independiente. Cada disco opera con un vector en forma independiente y esto permite que cada uno pueda satisfacer en forma individual solicitudes de E/S. Cada vez que graba debe actualizar las bandas de paridad.

RAID 5

Solo difiere de RAID 4 en que las bandas de paridad las distribuye a través de todos los discos.

RAID 6

Es un método que nació después de todos los enunciados anteriormente y consiste en tener un doble cálculo de paridad que se almacenan en bloques diferentes y de diferentes discos.

Bibliografía consultada

- Carretero Pérez, J., De Miguel Anasagasti, P., García Carballeira, F. y Pérez Costoya, F. (2001). *Sistemas Operativos: Una visión aplicada* (2ª ed.). Buenos Aires: Mc Graw Hill.
- Silberschatz, A. y Galvin, P. B. (2006). *Fundamentos de Sistemas Operativos* (7ª ed.). México: Addison Wesley Logman.
- Stallings, W. (2005). *Sistemas operativos: Principios de diseño e interioridades* (5ª ed.). Madrid: Pearson/Prentice Hall.
- Tanenbaum, A. S. y Van Steen, M. (2009). *Sistemas operativos modernos* (3ª ed.). México: Pearson/Prentice Hall.